Случайность и вероятность

Случайность и вероятность

Лут из врагов, критические удары, разброс урона, процедурная генерация, непредсказуемость ИИ - случайность делает игру живой. Банальный `random()` может испортить впечатление: 10 промахов подряд при шансе 70%, ни одного редкого предмета за час игры, одинаковые уровни при разных запусках.

Как использовать случайность **правильно**?

Псевдослучайность и seed

Компьютер не умеет генерировать "настоящую" случайность. `random()` - это **детерминированная формула**, которая выдаёт последовательность чисел, выглядящих случайными. Следующее число вычисляется из предыдущего.

Начальное значение - **seed** (зерно). Один и тот же seed -> одна и та же последовательность. Всегда.

Python C#

```python
import random

random.seed(42)
print(random.random())  # 0.6394...
print(random.random())  # 0.0250...
print(random.random())  # 0.2754...

random.seed(42)  # reset
print(random.random())  # 0.6394... - same sequence
print(random.random())  # 0.0250...
print(random.random())  # 0.2754...
```

Ок, зачем это нужно?

- **Процедурная генерация**: seed мира в Minecraft - одно число, из которого рождается весь мир. Тот же seed - тот же мир.
- **Реплей**: сохраняем seed + список команд -> можно воспроизвести партию.

- **Тестирование**: фиксируем seed -> баг воспроизводится каждый раз. Без этого - "иногда падает, иногда нет".

Если не задать seed, он обычно берётся из системного времени. Поэтому каждый запуск даёт разные числа. Для отладки **всегда** фиксируйте seed.

Взвешенный случайный выбор

`random()` даёт **равномерное** распределение - каждое значение одинаково вероятно. Но для лут-таблицы это не подходит: легендарный меч не должен падать так же часто, как зелье здоровья.

Нужен **взвешенный выбор**: каждый предмет имеет "вес" - чем больше вес, тем выше шанс.

Идея: веса складываются в **кумулятивный массив**. Бросаем случайное число и ищем, в какой отрезок оно попало:

```
Предмет:    Зелье(60)  Меч(25)  Щит(10)  Лук(5)
Кумулятив:  [60,       85,      95,      100]

random() * 100 = 73.2  →  попало в отрезок [60, 85)  →  Меч
```

Для поиска позиции - `bisect.bisect()` в Python, `List.BinarySearch()` в C#. Оба находят нужный отрезок за **O(log n)** вместо линейного перебора. Остаётся обернуть это в класс с методами `add(item, weight)` и `roll()`.

Знакомый паттерн: одна и та же таблица, разные веса - разные враги роняют разный лут:

Враг	Зелье	Меч	Редкий	Легендарный
Слизень	90	10	0	0
Рыцарь	40	35	20	5
Босс	10	20	40	30

Проблема полос

Хм. Шанс критического удара - 25%. Игрок бьёт 10 раз и ни одного крита. Баг? Черная полоса? Нет - математика.

Вероятность **не** получить крит за один удар: 75%. За 10 ударов подряд: $0.75^{10} \approx$ **5.6%**. Т.е. каждый ~18-й игрок увидит 10 промахов подряд. При тысячах игроков - это постоянные жалобы на "сломанный рандом".

А при 50% шансе? Вероятность 8 промахов подряд: $0.5^8 \approx 0.4\%$. Редко, но за час игры с сотнями бросков - вполне реально.

Проблема не в генераторе. Проблема в том, что **равномерное распределение не гарантирует отсутствие полос**. Люди ожидают "равномерное распределение по времени", а получают "равномерное распределение в бесконечности".

Два решения: PRD и shuffle bag.

Псевдослучайное распределение (PRD)

Подход из Dota 2 и других игр от Valve. Идея: вместо фиксированного шанса используем **нарастающий**. После каждого промаха шанс увеличивается, после успеха - сбрасывается.

Ядро механики - метод `check()` :

```
check():
    if random() < current_chance:
        current_chance = C    // reset - success
        return true
    current_chance += C       // miss - increase chance
    return false
```

Вся сложность - в нахождении **C** по номинальному шансу. Если номинальный шанс 25%, C ≈ 0.085. Связь нелинейная: нужно решить уравнение "при данном C среднее число попыток до успеха = 1/p". Это делается бинарным поиском - перебираем C, для каждого считаем ожидаемый средний шанс, ищем такой C, где средний шанс совпадает с номинальным.

Для ленивых - таблица готовых значений:

Номинал	C
5%	0.003
10%	0.015
15%	0.038
20%	0.055
25%	0.085
30%	0.115
50%	0.302

Посмотрим, как это работает при номинальном шансе 25% (C ≈ 0.085):

Удар #	Шанс	Пояснение
1	8.5%	Первый удар - шанс ниже номинала
2	17.0%	Не попал -> шанс растёт
3	25.5%	Ещё промах -> растёт
4	34.0%	...
5	42.5%	К пятому удару шанс уже высокий
8	68.0%	Почти гарантированно
12	100%	Максимум - **обязательно** сработает

Средний шанс по-прежнему ~25%. Но максимальная полоса промахов **ограничена** - примерно 1/C ударов. Игрок не заметит разницы в среднем, но не столкнётся с "10 промахов подряд".

PRD меняет **распределение**, не среднее значение. Средний шанс крита остаётся 25%, но ощущается "честнее". Именно поэтому Dota 2, Warcraft III и многие другие игры используют этот подход вместо чистого `random`.

Мешок без возврата (Shuffle Bag)

Другой подход к борьбе с полосами - **shuffle bag** (мешок с перемешиванием). Принцип как колода карт: кладём в мешок N предметов, перемешиваем, достаём по одному. Когда мешок пуст - наполняем заново.

```
ShuffleBag(["crit", "normal", "normal", "normal"]):

draw():
    if bag is empty:
        bag = copy of original
        shuffle(bag)          // Fisher-Yates
    return bag.pop()

Round 1: [normal, crit, normal, normal] → pop → pop → pop → pop
Round 2: [crit, normal, normal, normal] → pop → pop → pop → pop
// every 4 draws: exactly 1 crit, order random
```

В Python `random.shuffle()` уже реализует Fisher-Yates. В C# придётся написать цикл самим: проходим массив с конца, каждый элемент меняем со случайным из оставшихся.

Получаем гарантию: каждые 4 атаки - ровно 1 крит. Не "в среднем 1", а **ровно 1**. Порядок внутри четвёрки случайный, но распределение идеальное.

Shuffle bag используется для:

- Крит/промах, где важна "честность"
- Spawn волн врагов - каждый тип появится ровно N раз за волну
- Тетрис - следующая фигура выбирается из мешка из 7 фигур (каждая ровно раз), потом новый мешок

Fisher-Yates shuffle - алгоритм перемешивания за O(n). Проходим массив с конца, каждый элемент меняем с случайным из оставшихся. Равномерное распределение перестановок гарантировано математически.

PRD vs Shuffle Bag: PRD сохраняет элемент неожиданности (крит может выпасть на 1-м или на 10-м ударе). Shuffle Bag жёстче - за N попыток **ровно** K успехов. Выбирайте в зависимости от того, что важнее: ощущение случайности или строгая гарантия.

Нормальное распределение

Ну, допустим, урон 100. Но хочется, чтобы иногда было 95, иногда 108, изредка 80 или 120. Равномерный рандом `[80, 120]` даст 80 так же часто, как 100 - неестественно. Нужен **bell curve**: большинство значений у центра, экстремальные - редко.

Подход 1: сумма кубиков

Самый простой способ получить колокол - **бросить несколько кубиков и сложить**. Настольщикам это знакомо: 3d6 даёт совсем другое распределение, чем 1d18.

```
1d12:   все значения 1-12 равновероятны
2d6:    7 выпадает в 6 раз чаще, чем 2 или 12
3d6:    10-11 выпадает в ~12 раз чаще, чем 3 или 18
```

Чем больше кубиков, тем уже колокол. Это **центральная предельная теорема** в действии: сумма независимых случайных величин стремится к нормальному распределению. Для игрового урона 2-3 "кубика" уже достаточно:

```
damage = base + sum(random(-spread, spread) for i in range(3)) / 3
```

Просто, понятно, не нужна никакая математика сверх `random()`.

Подход 2: gauss напрямую

Если нужен точный контроль - генерируем множитель из нормального распределения с центром в 1.0 и стандартным отклонением `spread`. Умножаем базовый урон на множитель. Обязательно **clamp** - без него результат может улететь:

```
multiplier = gauss(mean=1.0, std=spread)
multiplier = clamp(multiplier, 0.5, 1.5)
damage = base * multiplier
```

Можно использовать **преобразование Бокса-Мюллера** (Box-Muller transform): из двух равномерных случайных чисел получаем одно нормально распределённое:

```
u1, u2 = random(), random()
normal = sqrt(-2 * ln(u1)) * cos(2π * u2)
```

Три сигмы

`spread` - стандартное отклонение (σ). При `spread=0.15` и базовом уроне 100:

- ~68% значений попадут в ±1σ: [85, 115]
- ~95% - в ±2σ: [70, 130]
- ~99.7% - в ±3σ: [55, 145]

Это правило "трёх сигм" - знакомое из курса теории вероятностей. На практике: `spread=0.1` - почти стабильный урон, `0.2` - ощутимый разброс, `0.3+` - хаос.

Где ещё пригодится

Нормальное распределение - не только для урона:

- **Генерация NPC**: рост, вес, характеристики. Большинство "средних", единицы - экстремальные.
- **Разброс выстрелов**: пули летят не точно в прицел, а с отклонением. Нормальное распределение даёт плотный центр и редкие промахи.
- **Время появления**: враги спавнятся "примерно каждые 5 секунд", но не ровно - `gauss(5.0, 0.5)` .
- **Генерация лута**: вместо фиксированного "меч +10" - `gauss(10, 2)` , т.е. от +6 до +14, чаще +9..+11.

Clamp обязателен. Без него `gauss` может вернуть множитель 0.1 или 3.0 - редко, но разрушительно. Представьте урон 10 или 300 при базе 100. Сумма кубиков этой проблемы лишена - у неё есть естественные границы.

Управление seed

В сложной игре мы используем случайность везде: генерация уровня, лут, ИИ, разброс урона, частицы. Если все берут числа из одного генератора, то порядок вызовов влияет на результат. Игрок развернул камеру - частицы запросили 50 случайных чисел - лут с босса изменился.

Решение: **отдельный генератор на каждую подсистему**.

Один **мастер-генератор** порождает seed'ы для подсистем. Каждая подсистема - отдельный экземпляр `Random` :

```
master = Random(12345)

world_gen  = Random(master.next())   // level generation
loot       = Random(master.next())   // loot tables
combat     = Random(master.next())   // damage spread, crits
particles  = Random(master.next())   // visual effects (doesn't affect gamepla
```

В Python - `random.Random(seed)` создаёт независимый генератор. В C# - `new Random(seed)` . Главное: подсистемы не делят один генератор, поэтому частицы не влияют на лут.

Теперь частицы могут запрашивать сколько угодно случайных чисел - это не повлияет на лут и генерацию мира. А при фиксированном `master_seed` вся игра **детерминирована**: тот же seed -> тот же мир, тот же лут, те же исходы боёв.

Это критично для:

- **Реплея**: seed + команды = полное воспроизведение партии
- **Отладки**: баг воспроизводится, потому что seed зафиксирован

- **Спидрана**: игроки ищут "хорошие" seed'ы с удачной генерацией
- **Соревновательных игр**: все игроки получают одинаковый уровень

Без этого, может спидраннеры и будут благодарны за такие особенности, но не факт.

От лут-таблиц до нейросетей

Случайность - одна из основ машинного обучения:

- **Распределения вероятностей** - всё, что мы обсуждали (равномерное, нормальное, взвешенное), является фундаментом теории вероятностей. В ML модели оперируют распределениями напрямую.
- **Monte Carlo методы** - MCTS из минимакса симулирует случайные партии для оценки позиции. Чем больше симуляций - тем точнее оценка. Это фундаментальный метод: случайная выборка для аппроксимации сложных вычислений.
- **Стохастический градиентный спуск (SGD)** - обучение нейросетей использует случайные подвыборки данных. Случайность ускоряет обучение и помогает избежать локальных минимумов.
- **Exploration vs exploitation** - агент в reinforcement learning балансирует между "использовать лучшее известное действие" и "попробовать случайное". Та же проблема, что и в roguelike: идти проверенным путём или рискнуть?

Генераторы псевдослучайных чисел (ГПСЧ) - тема, которая тянется от теории чисел до криптографии. `random()` в Python использует Mersenne Twister - алгоритм с периодом $2^{19937} - 1$. Для игр этого достаточно. Для криптографии - нет.

Итого

Подход	Когда использовать
`random()` (равномерный)	Простые случаи, позиции, углы
Взвешенный выбор	Лут-таблицы, спавн, выбор из вариантов
PRD	Шанс крита/уклонения - когда полосы раздражают
Shuffle Bag	Тетрис, волны врагов - когда нужна строгая гарантия
Нормальное распределение	Разброс урона, генерация характеристик
Отдельные seed	Всегда, если нужна детерминированность

Что	Где подробнее
Seed для генерации уровней	Процедурная генерация
Шум Перлина - "гладкая случайность"	Шум Перлина
Реплей и детерминизм	Пошаговые игры
Monte Carlo в деревьях игр	Минимакс
Случайные параметры врагов	ИИ врагов
Разброс частиц	Системы частиц
Эволюция параметров через отбор и мутацию	Генетические алгоритмы
Фиксированный seed для отладки	Отладка
SGD и обучение нейросетей	Нейросети для игр
Вероятностные переходы, генерация имён	Цепи Маркова
Стохастические правила роста	L-системы

Powered by Wiki.js